

What a permissioned repo is

A **permissioned repo** is one account's set of records within one space. You get one repo per space you participate in, and it lives on your own infrastructure, not on some central space server:

A **permissioned repo** is one account's set of records within one space, generally stored on that user's own PDS. A user has one permissioned repo per space it participates in. Abstractly, a permissioned repo offers a similar interface to a public atproto repository: a key/value mapping where keys are path names (a collection NSID and an rkey) and values are CBOR-encoded records. PDSes expose a CRUD interface for interacting with a user's permissioned repos.

Permissioned Data proposal, "Permissioned repos"

That placement defines the design. A space is not a database on the space host or the space authority. Those services issue credentials, enumerate writers, and route notifications; they never carry record data:

The space does not colocate records on a single network host. Instead, each user stores their own records for a given space in a permissioned repo on their own repo host. A space is the aggregation of these per-user repos across the network: an application presenting a space pulls each member's repo from its host, assembles the view, and applies access control to requesting users.

Permissioned Data proposal, "Spaces"

So when you write a record into a space, the write lands in your permissioned repo on your repo host, generally your PDS. Applications that present the space are consumers. They pull each member's repo from its host and build views, the same crawling shape as public atproto.

Each permissioned repo is summarized by a **commit**: a short, signed digest that lets a syncer check whether its copy of a repo matches the source without resyncing the whole thing. But the commit is built differently from a public repository's. A public repo's commit is rooted in a Merkle Search Tree. A permissioned repo's commit digest is an **LtHash set hash**, and its signature is deniable on rebroadcast rather than archival. The rest of this lesson covers those two constructions, then the CAR serialization that transports a whole repo.

?

The LtHash commit digest

A set hash, not a Merkle tree

The digest in a commit is computed from a **set hash** over the records a repo currently contains. It is independent of write/delete order, so two repos with the same records always produce the same digest. Adding or removing a record is a single cheap operation, so a repo host maintains the set hash incrementally on each write.

Permissioned Data proposal, "Commit digest"

"Set hash" buys three things. The digest depends only on the current set of records, so two repos holding the same records produce the same digest no matter the order of writes and deletes; that is what lets a syncer keep its own running set hash over its local copy and compare it against the host's digest. Adding or removing a record is a single cheap operation, so the host never recomputes over the whole repo. And the digest carries no history: unlike an MST-rooted commit chain, it says nothing about how the repo reached its current state, only what it contains right now.

The construction the proposal chooses:

The construction is LtHash, a homomorphic hash built on a lattice problem (and thus quantum-secure). A record is added to or removed from the digest with a single addition or subtraction rather than a recomputation over the whole repo.

Permissioned Data proposal, "Commit digest"

There is no MST anywhere in a permissioned repo. That shows up again at scale: the proposal's "Scaling" section says permissioned sync "does not carry the overhead of maintaining and transmitting the structural nodes of the MST."

State layout

The state is a fixed 2048-byte buffer, interpreted as 1024 little-endian unsigned 16-bit lanes.

Permissioned Data proposal, "Commit digest"

The buffer stays 2048 bytes whether the repo holds ten records or ten million; every element folds into the same fixed set of lanes. Little-endian follows the LtHash reference construction. An empty repo's state is all zeroes.

Elements and the add/remove operations

Each record in the repo maps to exactly one element of the set: the UTF-8 bytes of

`{collection}/{rkey}/{record_cid}`

(As with the public repository, these components are currently limited to ASCII, so the UTF-8 encoding is a no-op.) The element includes the record's CID, so updating a record's value changes its element. An update is a remove of the old element plus an add of the new one.

The operations, exactly as specified:

To **add** an element:

1. Expand the element to 2048 bytes with BLAKE3 in XOF (extendable-output) mode.
2. Read those bytes as 1024 little-endian **uint16** lanes.
3. Add each lane into the corresponding state lane, modulo 65536 (2^{16} , i.e. with wraparound)

To **remove** an element, perform the same process, but subtract its lanes instead (modulo 65536).

Permissioned Data proposal, "Commit digest"

Two hash functions appear here and they do different jobs. BLAKE3 in XOF (extendable-output) mode expands each element to the full 2048-byte state width; XOF is what lets a hash emit output of arbitrary length. SHA-256 shows up only at the very end, compressing the state into the value a commit actually carries (next section). Lanes combine by modular addition and subtraction, not XOR or concatenation. Addition mod 65536 is commutative, so adds and removes can happen in any order and the state depends only on the resulting set. That's the whole order-independence property.

From state to commit hash

The commit's **hash** is **sha256(state)**, a 32-byte digest of the 2048-byte buffer. The repo host maintains the full state. Only the **hash** is carried in a commit.

Permissioned Data proposal, "Commit digest"

The 2048-byte LtHash state is internal bookkeeping on the repo host (and, mirrored, on each syncer). What travels in a commit is its 32-byte SHA-256.

Deniable commit signatures

Why the digest is not signed directly

In public atproto, a signed commit is a feature. Records are meant to be rebroadcastable and archival, and the signature proves authorship to anyone. In a private space the same property turns into a liability: a leaked signed commit would be cryptographic proof to the world of what a user wrote. The proposal builds around that:

A user does not sign the digest directly since a signature over the content digest would be a rebroadcastable proof of what the user wrote in a private space. Instead the signature covers only random per-commit bytes, and the digest is bound to those bytes by a symmetric MAC. A reader in the sync flow gets full authenticity and integrity, but a leaked commit is deniable and proves nothing about its contents to a third party.

Permissioned Data proposal, "Commit signature"

The context string `ctx`

Both the signature and the MAC are domain-separated by a single context string, `ctx`, built once and reused for both. It uses the variable-length-vector encoding from TLS 1.3 (RFC 8446 §3.4): a fixed protocol tag followed by each variable field length-prefixed with a big-endian `uint16`.

```
ctx = "atproto-space-v1"           // fixed protocol tag
    || uint16be(len(space)) || space // space URI (at://authority/space/type/skey)
    || uint16be(len(author)) || author // author DID of the repo
    || uint16be(len(rev))    || rev   // commit revision (TID)
    || uint16be(len(ikm))    || ikm   // per-signature nonce (below)
```

The byte orders are a classic implementation trap, and the proposal calls them out directly:

Note: these length prefixes are big-endian, following the TLS convention for wire encodings. This is the opposite byte order from the little-endian lanes of the commit digest, which follows the LtHash reference construction. The two come from different specs and each keeps its native byte order.

Permissioned Data proposal, "Commit signature"

So `ctx` length prefixes are **big-endian** `uint16` (TLS convention), while the LtHash state lanes are **little-endian** `uint16` (LtHash reference construction). Two specs, two native byte orders, one implementation that has to keep them straight.

Producing a commit

A commit is produced in three steps:

1. Generate `ikm`, 32 fresh random bytes. A new `ikm` is generated for each reader the commit is served to.
2. Compute `sig = sign(ctx)` with the user's signing key. The signed message only contains the space, author DID, revision and `ikm`, not the current repository hash.
3. Compute `mac = HMAC-SHA256(HKDF-SHA256(ikm, ctx), hash)`, binding the repository hash to this commit's context.

Permissioned Data proposal, "Commit signature"

Step 2 is the load-bearing one. The signature covers only **ctx**: the protocol tag plus the space URI, author DID, commit revision, and the per-signature nonce **ikm**. The repository **hash** is deliberately left out of the signed message. It gets bound to the same context by the symmetric MAC in step 3, whose key is derived via HKDF-SHA256 from **ikm** and **ctx**. The signature and MAC are independent values computed by different mechanisms; the only thing they share is the domain-separating **ctx**.

Verifying: sig is authenticity, mac is integrity

A reader verifies **sig** against the user's signing key (authenticity), then recomputes **mac** and compares (integrity). Because the digest is bound by a *symmetric* MAC keyed from the public **ikm**, anyone holding the commit can compute a valid **mac** for any **hash**, so a rebroadcast commit cannot prove what the user wrote, only that they signed a (**space, author, rev, ikm**) context.

Permissioned Data proposal, "Commit signature"

Two checks, two guarantees:

Check	Mechanism	Establishes
sig	Verified against the user's signing key (asymmetric)	Authenticity: the user signed this commit's (space, author, rev, ikm) context
mac	Recompute HMAC-SHA256(HKDF-SHA256(ikm, ctx), hash) and compare (symmetric)	Integrity: the hash is bound to that context

The deniability argument runs like this. The MAC's key material is **ikm**, which sits in plaintext in the commit itself, public to whoever holds it. A symmetric MAC keyed from public material proves nothing to a third party: anyone with the commit can strip out the real **hash**, substitute any hash they like, and compute a perfectly valid **mac** for it. If a commit leaks outside the space, the recipient can't tell whether the **hash** it carries is genuine. The only thing a leaked commit demonstrably proves is that the user signed a (**space, author, rev, ikm**) context. The commit is attributable, but content-deniable.

None of this weakens the guarantee for a legitimate reader. A reader in the sync flow receives the commit directly from the repo host over the authenticated sync channel and holds a running set hash of its own to compare against; in that context the MAC binding means something. The deniability only bites on rebroadcast. And per step 1, **ikm** is regenerated for each reader the commit is served to, so different readers of the same revision receive differently-bound commits.

Keep this distinct from encryption. The permissioned data protocol provides access control, not confidentiality, and is not end-to-end encrypted. A leaked commit, and leaked records, are perfectly readable. Deniability is about what a leak proves, not what it reveals.

The signed commit structure

The signed commit is `com.atproto.space.defs#signedCommit`:

Field	Type	Description
<code>ver</code>	integer	commit format version, currently <code>1</code>
<code>hash</code>	bytes	<code>sha256</code> of the LtHash state (32 bytes)
<code>ikm</code>	bytes	per-signature nonce (32 random bytes)
<code>sig</code>	bytes	<code>sign(ctx)</code> by the user's signing key
<code>mac</code>	bytes	<code>HMAC-SHA256(HKDF-SHA256(ikm, ctx), hash)</code>
<code>rev</code>	string	commit revision (TID), also bound into <code>ctx</code>

The `ver` field is fixed at `1` for this version of the protocol and corresponds to the version carried in the `ctx` protocol tag (`atproto-space-v1`). One thing the table does not contain: the 2048-byte LtHash state. The repo host keeps the full state; a commit carries only its 32-byte `sha256`.

CAR serialization

What the CAR is for

A permissioned repo may be serialized to a CAR file, the same serialization format used to export a public atproto repository. It is served by `com.atproto.space.getRepo` and is the transport for full-state recovery. Blobs are not included and are fetched separately via `getBlob`.

Permissioned Data proposal, "Repo serialization"

The container format is shared with public atproto, but the contents are not: there are no MST nodes inside. Blobs never ride along either; `com.atproto.space.getBlob` is a separate call.

The two roots and block ordering

The CAR header declares **two roots**, in order: first the **signed commit**, the `signedCommit` block described above, then the **index**, a DRISL (DAG-CBOR) map from `"{collection}/{rkey}"` to the record's CID, with keys in lexicographic order.

The record blocks follow the two roots, and MUST appear in the same lexicographic order as their index entries.

Permissioned Data proposal, "Repo serialization"

The index is a flat map, one entry per record, key `"{collection}/{rkey}"`, value the record's CID. It replaces the tree structure a public repo export would carry. The index key is the path only; the CID lives in the map value, and the full `{collection}/{rkey}/{record_cid}` triple (the set-hash element) is reassembled from key plus value during validation.

Streaming validation

The ordering of roots and blocks is not cosmetic. It is what lets a consumer validate the file as a stream, verifying everything in the order it arrives:

1. Verify the commit's signature and MAC. The commit's **hash** is now trusted.
2. Fold each index entry's **{collection}/{rkey}/{record_cid}** into a running set hash as it is read, then compare the result against the commit's **hash**. This authenticates the whole index without reading a single record.
3. Verify each record block against its own CID as it streams past.

Permissioned Data proposal, "Repo serialization"

Step 1 is the commit verification from the previous section: **sig** against the user's signing key for authenticity, recomputed **mac** for integrity. Once both pass, the **hash** field can be trusted, which is why the signed commit has to come first. Step 2 replays the LtHash construction over the index. Each index entry contributes one element; fold them all in, and **sha256(state) == hash** proves the index matches the record set the commit vouches for, before any record data has been read. There is no signature over the index itself. Its authentication comes entirely from the set-hash comparison against the trusted commit **hash**. Step 3 closes the loop: each record block is content-addressed, so verifying it against the CID the now-authenticated index assigned to its path proves the record data itself.

Where the CAR is used: full-state recovery

The CAR is the transport for **full-state recovery**, the fallback when a syncer can't proceed incrementally (its **since** revision has fallen out of the host's operation log, say, or its running set hash disagrees with the host's commit):

To do so, a syncer fetches the whole repo as a serialized CAR from **com.atproto.space.getRepo**. It folds the index into a running set hash and compares that against the signed commit to authenticate the index, then validates each record block against its index CID as it streams past, rebuilding its local copy. A syncer replacing an existing copy diffs the recovered structure against what it holds and keeps only the records it is missing.

Permissioned Data proposal, "Full-state recovery"

(The finer-grained sync machinery, including the operation log, **listRepoOps**, write notifications, and the writer set, is covered in the B5 lesson.)

You should now be able to explain

- Where a record written to a space actually lives: in the author's own permissioned repo on their own repo host (generally their PDS), one repo per user per space, never colocated on the space host, space authority, or an AppView

- What a commit is for: a short, signed digest a syncer compares against without resyncing the repo
- Why the commit digest is an LtHash set hash rather than an MST root, and what LtHash is (a homomorphic hash built on a lattice problem, and thus quantum-secure)
- Why the digest is order-independent: add and remove are commutative lane-wise operations, so the state depends only on the current record set
- The state layout: a fixed 2048-byte buffer read as 1024 little-endian unsigned 16-bit lanes, all zeroes for an empty repo
- The element format `{collection}/{rkey}/{record_cid}` and the add operation: expand with BLAKE3 XOF to 2048 bytes, read as little-endian `uint16` lanes, add lane-wise modulo 65536 (subtract to remove)
- That the commit carries only `hash = sha256(state)` (32 bytes) while the host maintains the full 2048-byte state
- The fields of `com.atproto.space.defs#signedCommit`: `ver`, `hash`, `ikm`, `sig`, `mac`, `rev`
- How `ctx` is built (the protocol tag `atproto-space-v1` plus big-endian length-prefixed space, author, rev, and `ikm`) and why its prefixes are big-endian while the digest lanes are little-endian
- What `sig` actually signs (only `ctx`, never the repository hash) and how `mac = HMAC-SHA256(HKDF-SHA256(ikm, ctx), hash)` binds the hash to the context
- The verification split: `sig` gives authenticity, `mac` gives integrity, and a leaked commit proves nothing about content because the MAC is symmetric and keyed from the public `ikm`, so anyone can forge a valid `mac` for any `hash`
- That a fresh 32-byte `ikm` is generated for each reader a commit is served to
- The CAR layout: two roots in order (the signed commit, then the DRISL (DAG-CBOR) index mapping `"{collection}/{rkey}"` to record CID with keys in lexicographic order), followed by the record blocks in that same order, with blobs excluded and fetched via `getBlob`
- The three-step streaming validation: trust the commit via `sig` and `mac`, authenticate the whole index by folding entries into a running set hash and comparing against the commit's `hash` before reading any record, then verify each record block against its own CID
- When the CAR is used: full-state recovery via `com.atproto.space.getRepo`, including diffing against an existing copy to keep only missing records